Performance assessment of grid-tdse

Evaluation efficiency and parallelization on the Sigma2 clusters Saga, Betzy and Olivia

5 November 2026. Prepared by Claude Code (Claude Opus 5.5).
Code state: grid-tdse `d3b357e`, integrator-lib `ab5332f`, grid-lib `8950c3a`.
Status: initial assessment. No repository code was changed, and nothing was run on the clusters.

Summary

The largest available gains come from the angular representation and the local kernels, not from more MPI ranks. On the Phase 9g problem at $l_{\max} = 8$, one Hamiltonian application (an *H action*) falls from 473 ms to 23 ms on one core with no approximation, and to 7 ms if the total angular momentum is truncated at $L \leq 4$ (Table 1). With a fourth-order propagator as well, a full run on this problem at similar dipole accuracy is about 40 times faster with no truncation (Section 2.9).

On the clusters, memory is the first blocker. At $l_{\max} = 16$ each rank retains 9 GB to hold an 11 MB tile of the state, mostly in global operator arrays that the distributed action never uses. Betzy provides 1.9 GiB per core, so this case cannot run there today. Betzy's minimum job of 512 cores also needs a grid with at least 512 element tiles, whereas the Phase 9g grid stops at 16. Modest changes address both (Section 3).

`one_radial` has no time propagation yet, and its ground-state path uses dense diagonalization. The existing Davidson solver with a sparse H action solves the $l_{\max} = 60$ case more than 1000 times faster, and the problem does not need a large cluster (Section 4).

Table 1: Time per H action at $l_{\max} = 8$ on the Phase 9g grid, one core. Each row adds to the one above. The last column is the weighted L^2 norm of the final-state difference from production.

Cumulative change	ms per H action	Speed-up	Difference from production
Production	473	1	
Lean Krylov loop	390	1.2	1.2×10^{-15}
+ Numba kernels	116	4.1	2.0×10^{-15}
+ coupled angular basis, all L	23	21	1.3×10^{-12} (checked at $l_{\max} = 6$)
+ truncation at $L \leq 8$	17	28	depends on the pulse (Table 5)
+ truncation at $L \leq 4$	7.2	66	depends on the pulse
+ truncation at $L \leq 2$	2.9	164	weak pulses only

Confidence. All timings come from one AMD Ryzen 9 3900X workstation with shared-memory MPI; a Saga core is about 2.8 times slower. Nothing has run on a cluster, so network cost is unmeasured (Section 7).

Recommended order (Section 6). Quick fixes, Numba kernels, the fourth-order propagator and `one_radial`, none of which depends on the decision below. Then a Saga benchmark before Betzy or Olivia.

Decision needed. Should the code adopt the coupled angular basis? The plan currently retains the uncoupled one. The coupled basis is the largest single gain, with 17 times fewer Coulomb couplings at $l_{\max} = 8$ and 35 times fewer at $l_{\max} = 16$ with no truncation, and it is also the largest piece of work, at several weeks.

1 Scope and method

Test problem. Unless noted otherwise, tests use H_2^+ in the `two_radial` representation on the Phase 9g grid: the electronic coordinate r has 4 elements of 11 points on $[0, 16]$, and the internuclear coordinate R has 4 elements of 21 points on $[0, 8]$, giving $n_r \times n_R = 39 \times 79$. The total magnetic quantum number is $M_{\text{tot}} = 0$. The pulse is the Phase 9g one: $E_0 = 0.02$, $\omega = 2\pi$, one cycle, $T = 1$ a.u. Initial states are the archived Phase 9g ground states.

Terms.

- *H action*: one application of the full Hamiltonian, including the field term, to a state. Times given as ms/H are all-in, that is, the wall time of the propagation divided by the number of H actions, so they include Krylov overhead.
- *Channel*: one angular index of the compact basis, (l_1, l_2, m) in production and (l_1, l_2, L) in the coupled basis.
- *Coupling*: one entry (output channel, input channel, multipole λ) of the Coulomb table. The Coulomb cost is proportional to couplings times radial points.
- *Efficiency*: $T_1/(P\,T_P)$ on P ranks.
- *Production* is the code at the commits above. *Prototype* is separate benchmark code that reuses production's grids, radial multipoles and coupling data, and replaces the kernels, the Krylov loop, or the angular basis. The prototypes and raw outputs are not in the repository.

Measurement conditions. The machine is an AMD Ryzen 9 3900X (12 cores, 24 threads) with one thread per process unless noted, Python 3.14, NumPy 2.5.4, SciPy 1.18.1 and Numba 0.67. The Phase 6 Hamiltonian workload runs 2.7–2.9 times slower on a Saga Gold 6138 core than here, and a factor of 2.8 is used wherever a Saga time is projected. Another workload occupied about 16–18 logical CPUs for part of the assessment, and tabulated timings from that period were repeated once it had ended. The strong-scaling and thread series used a load gate: a run counted only if CPUs outside its own stayed below a combined load of 0.8 CPUs. All 48 gated runs passed.

2 `two_radial`: efficiency of evaluations

File references use the abbreviations `two_radial/` for `grid_tdse/spherical/two_radial/`, `one_radial/` for `grid_tdse/spherical/one_radial/`, and `krylov.py` for `integrator_lib/krylov.py` in integrator-lib. Line numbers refer to the commits on the first page.

2.1 Where the time goes

Table 2 shows the production cost. The Coulomb term dominates at high $l_{\max}$, because couplings grow as about l^6 (from 105,441 at $l_{\max} = 8$ to 7,744,055 at $l_{\max} = 16$). Radial work and the Krylov loop dominate at low $l_{\max}$. A Python floor of about 570 µs per H action limits very small tiles. This corresponds to 1.5–1.7 ms on a Saga core, the same magnitude as the 1.66 ms per action measured at $l_{\max} = 2$ on 8 ranks in Phase 6.

2.2 Krylov loop

The short iterative Lanczos path does more work per iteration than it needs:

- Every iteration computes a matrix exponential, assembles the full approximate state, and takes its norm for the convergence test (`krylov.py:443`, `:446`, `:457`). State assembly is $O(m^2 N)$ per step for Krylov dimension m and state size N, and the norm adds a collective. The test can be done from the projected matrix, with the state built only on exit.

Table 2: Production cost on one core. Shares are of total runtime, from separate profiled runs. The last two columns give the share of runtime outside the H action, in production and in the lean Krylov loop of Section 2.2.

$l_{\max}$	Channels	Couplings	ms/H	Coulomb	Radial	Outside H	Outside H, lean
2	12	103	4.1	8%	38%	46%	18%
4	49	2,302	26.9	25%	27%	36%	20%
6	128	19,958	123	49%	17%	27%	12%
8	265	105,441	473	67%	9%	21%	7%

- Each state wrapper runs `isfinite` over its whole tile when it is constructed, and about 20–30 wrappers are created per H action (`two_radial/distributed_2d_state.py:439`).
- The right-hand side is multiplied by $-i$ (`grid_tdse/real_time/propagation.py:602`) and then divided by the same factor (`krylov.py:399`).
- The real-time integrator is built without `inner_products` (`propagation.py:682`), so each projection is a separate scalar reduction. In a traced run, 108 of 110 Allreduce calls were scalar. The count is $m(m-1)/2 + 4m + 2$ per step, which is 8–13 per H action at the dimensions seen here.

The lean prototype uses a contiguous basis array, one weight multiplication per iteration, batched projections, a tridiagonal eigensolver, and a final state built on exit. It needs 2 collectives per H action. With reorthogonalization kept, the state agrees with production to 3.8×10^{-15} or better. Removing reorthogonalization left the H-action counts unchanged and the states within 3.6×10^{-15} at the Krylov dimensions tested (up to about 15, on short runs). The 390 ms/H in Table 1 is without reorthogonalization; with it, the figure is 404 ms/H. Longer runs at $m = 20$ should be checked before changing the default.

2.3 Local kernels

The Coulomb kernel (`two_radial/hamiltonian.py:384`) costs 0.96 ns per coupling and radial point. Table 3 gives the cumulative effect of four changes, all within Numba. Hand-written C compiled with clang reached a factor of 4.0 at best, against 3.85 for Numba in the same run, so a C rewrite is not justified.

Table 3: Coulomb kernel on one core: cumulative speed-up against production. The maximum absolute difference in the action is at most 1.8×10^{-13}.

Cumulative change	ms, $l_{\max} = 8$	$l_{\max} = 6$	$l_{\max} = 8$	$l_{\max} = 10$
Production	313	1	1	1
Plain loop index, which removes the wraparound check that `start+point` forces	243	1.30	1.29	1.27
Row-wise slices	241	1.32	1.30	1.29
Real view of the complex arrays, since all couplings and radial multipoles are real	132	2.41	2.37	2.24
Grouping by λ: sum inputs first, then multiply by the radial multipole once	98.5	3.14	3.18	2.98
Radial tile as the outermost loop (tile 512)	81.5	3.79	3.84	3.92

The radial kernels (`two_radial/distributed_2d_hamiltonian.py:694`) rely on a Python loop with `take`, `moveaxis` and `einsum`. A Numba version with the row as the outer loop falls from 40–59 to 17–19 ns per coefficient, a factor of 2.3–3.1, with differences of at most 1.8×10^{-12}. The dipole action has a Python double loop per H action (`two_radial/dipole.py:188`) that should move into a kernel as well.

The Coulomb gain holds with all cores busy. With 12 pinned processes each working on its own tile, the speed-up was 3.5–4.4 across the three cases tested, and at $l_{\max} = 8$ on the 39×79 tile the efficiency rose from 0.68 to 0.79. With the Coulomb, radial and dipole kernels together, the all-in time per H action at $l_{\max} = 8$ is 116 ms against 390 ms.

2.4 Coupled angular basis

Production stores $M_{\text{tot}} = 0$ products $|l_1 m;\, l_2 {-} m\rangle$. The alternative combines them with Clebsch–Gordan coefficients:

$$|l_1 l_2 L\rangle = \sum_m \langle l_1 m\, l_2 {-} m \,|\, L0\rangle \; |l_1 m;\, l_2 {-} m\rangle. \tag{1}$$

This is a unitary transformation within each (l_1, l_2) block, so the radial one-body operators, which depend only on l_1 and l_2, do not change. Three exact properties then reduce the work for a z-polarized field:

- The Coulomb operator $\sum_\lambda f_\lambda(r, R)\, P_\lambda(\cos\theta_{rR})$ is a scalar under joint rotations, so it is block-diagonal in L.
- Reflection in a plane containing the z axis acts on $M = 0$ states as $(-1)^{l_1+l_2+L}$. The Coulomb term and a z-polarized field both preserve it, so a state that starts with $l_1 + l_2 + L$ even (natural parity) stays there. Parity alone reduces the couplings at $l_{\max} = 8$ from 105,441 to 37,787.
- The dipole is a rank-one tensor with zero projection, which gives $\Delta L = \pm 1$ only.

The ground state has $L = 0$, and higher L are reached only through repeated field interactions, so L can also be truncated with a controlled error (Section 2.5). Table 4 and Figure 1 show the effect on the table sizes. Between $l_{\max} = 8$ and 16, couplings grow as about $l^{6.2}$ in production, $l^{5.1}$ with all L, $l^{3.7}$ with $L \le 8$, and $l^{3.1}$ with $L \le 4$. Table 1 shows that the time per H action follows: from 116 ms to 23 ms at $l_{\max} = 8$ with no truncation.

Table 4: Channels / couplings for H_2^+ dynamics with $M_{\text{tot}} = 0$, natural parity, and the reduction in couplings against production in parentheses.

$l_{\max}$	Production	All L	$L \le 8$	$L \le 4$	$L \le 2$	$L = 0$
4	49 / 2,302	32 / 286 (8)	32 / 286 (8)	24 / 225 (10)	13 / 115 (20)	3 / 19
6	128 / 19,958	78 / 1,612 (12)	70 / 1,511 (13)	39 / 815 (24)	19 / 336 (59)	4 / 44
8	265 / 105,441	155 / 6,225 (17)	115 / 4,844 (22)	54 / 2,017 (52)	25 / 741 (142)	5 / 85
10	476 / 405,133	271 / 18,810 (22)	160 / 11,437 (35)	69 / 4,051 (100)	31 / 1,386 (292)	6 / 146
12	777 / 1,251,060	434 / 47,910 (26)	205 / 22,430 (56)	84 / 7,137 (175)	37 / 2,327 (538)	7 / 231
14	1,184 / 3,301,888	652 / 107,623 (31)	250 / 38,963 (85)	99 / 11,495 (287)	43 / 3,620 (912)	8 / 344
16	1,713 / 7,744,055	933 / 219,590 (35)	295 / 62,176 (125)	114 / 17,345 (446)	49 / 5,321 (1,455)	9 / 489

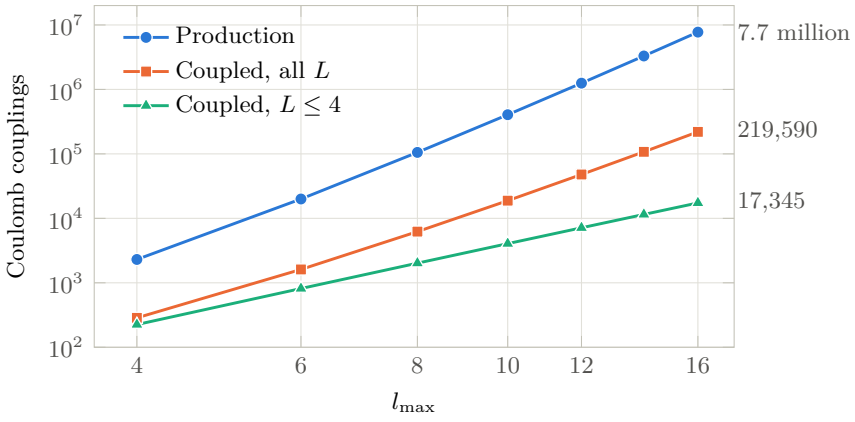


Figure 1: Coulomb couplings against $l_{\max}$. Values are in Table 4.

Validation. The prototype tables and dynamics passed these checks:

- Coulomb elements between different L or parity are at most 7.4×10^{-15} for H_2^+ up to $l_{max} = 10$, 1.8×10^{-15} for HD^+ up to $l_{max} = 6$ (which has odd multipoles), and 5.2×10^{-15} for two-electron atoms up to $l_{max} = 8$.
- Dipole elements outside $|\Delta L| = 1$ are at most 2.6×10^{-14} for H_2^+ up to $l_{max} = 8$ on both coordinates, and 1.0×10^{-14} for HD^+ up to $l_{max} = 6$. They were not checked for two-electron atoms.
- The archived Saga ground states have at most 1.1×10^{-14} weight at $L > 0$.
- Dynamics at $l_{max} = 6$ ($T = 1$, $\Delta t = 0.01$) took the same number of H actions as production (1,063 in length gauge, 1,129 in velocity gauge). The final states differ by 1.3×10^{-12} and 3.2×10^{-15}.

Reductions for the other systems are similar: at $l_{max} = 6$ HD^+ goes from 231 / 62,246 to 140 / 5,236 (12 times fewer couplings), and at $l_{max} = 8$ two-electron atoms go from 489 / 348,255 to 285 / 21,165 (16.5 times fewer).

Conditions and implementation. The reductions assume $M_{tot} = 0$, a z-polarized field, and an $L = 0$ start, as in the current runs. A field with other components couples M and loses the parity reduction. Observables that are not rotational scalars, such as $\langle \cos^2 \theta_R \rangle$, need new tables. Existing archives need converting. Absorbers act on radial coordinates only and should carry over, but this has not been tested. Kernels, layouts and MPI exchange only see a channel index, so they should need no structural change, although the coupled basis has so far run only on one rank. The prototype obtains its tables by diagonalizing L^2 within each (l_1, l_2) block; a production version could use the closed form in 6j symbols instead, which would also avoid the sympy Gaunt cost of Section 3.2.

2.5 Truncation in L

Table 5 shows how the L populations decay and what a cut costs. The final-state error tracks the square root of the population above the cut to within about 20%, which makes the population at the cut a usable run-time convergence monitor. These tests use $l_{max} = 6$, $\omega = 0.228$, two cycles, a 16 a.u. box and no absorber, so they do not establish the cut needed for strong-field ionization. The $E_0 = 0.10$ runs with lower cuts were not completed.

Table 5: Top: population in each L at the end of the pulse. Bottom: effect of a cut. Reference is the same run with all $L \leq 12$.

E_0	$L = 1$	$L = 2$	$L = 3$	$L = 4$	$L = 5$	$L = 6$	$L = 7$	$L = 8$
0.02	2.6×10^{-3}	4.0×10^{-5}	3.1×10^{-7}	1.1×10^{-8}	3.3×10^{-10}	7.7×10^{-12}	2.3×10^{-14}	4.8×10^{-16}
0.05	1.4×10^{-2}	1.5×10^{-3}	4.8×10^{-5}	8.9×10^{-6}	1.6×10^{-6}	2.0×10^{-7}	2.9×10^{-9}	3.9×10^{-10}
0.10	3.6×10^{-2}	2.0×10^{-2}	2.9×10^{-3}	6.1×10^{-4}	3.8×10^{-4}	1.1×10^{-4}	4.5×10^{-6}	1.4×10^{-6}

E_0	Cut	Population above cut	Its square root	Final-state error	Final $\lvert\delta z_e\rvert$
0.02	$L \leq 3$	1.2×10^{-8}	1.1×10^{-4}	1.1×10^{-4}	1.9×10^{-7}
0.02	$L \leq 4$	3.3×10^{-10}	1.8×10^{-5}	1.9×10^{-5}	1.0×10^{-8}
0.02	$L \leq 6$	2.3×10^{-14}	1.5×10^{-7}	1.5×10^{-7}	1.7×10^{-12}
0.05	$L \leq 4$	1.8×10^{-6}	1.4×10^{-3}	1.5×10^{-3}	2.5×10^{-5}
0.05	$L \leq 6$	3.3×10^{-9}	5.8×10^{-5}	6.2×10^{-5}	8.3×10^{-8}
0.10	$L \leq 8$	2.5×10^{-8}	1.6×10^{-4}	1.9×10^{-4}	4.1×10^{-7}

2.6 Fourth-order time propagation

Production uses the exponential midpoint rule, which is second order in Δt. For $H(t) = H_0 + f(t)V$, the fourth-order commutator-free Magnus scheme CF4 [1, 2] replaces one exponential of length Δt by two of length $\Delta t/2$, each an ordinary Krylov step with an effective field:

$$\Psi(t+\Delta t) = e^{-i\frac{\Delta t}{2}(H_0+f_2V)}\, e^{-i\frac{\Delta t}{2}(H_0+f_1V)}\, \Psi(t), \tag{2}$$

$$f_1 = 2\big[a_+ f(t_-) + a_- f(t_+)\big], \qquad f_2 = 2\big[a_- f(t_-) + a_+ f(t_+)\big], \tag{3}$$

with $t_\pm = t + (\frac{1}{2} \pm \frac{\sqrt{3}}{6})\Delta t$ and $a_\pm = \frac{1}{4} \pm \frac{\sqrt{3}}{6}$. The first-applied exponential must weight the earlier Gauss point. The reverse order silently falls back to second order, which happened in the first version of the prototype and shows up only in a convergence test.

One CF4 step costs the same as two midpoint steps of $\Delta t/2$, so CF4 at Δt compares directly with midpoint at $\Delta t/2$. At equal H actions the dipole error is 4,000 to 15,000 times smaller (Table 6, Figure 2). How many H actions this saves depends on the accuracy target.

Table 6: Midpoint against CF4 at $l_{\max} = 4$, length gauge, $E_0 = 0.05$, $\omega = 0.98$, $T = 19.2$. The error is $\max_t |\delta z_e(t)|$ against CF4 at $\Delta t = 0.01$, with $\max_t |z_e| = 0.11$. H/exp is H actions per exponential.

Midpoint				CF4			
Δt	H actions	H/exp	Error	Δt	H actions	H/exp	Error
0.04	9,526	19.8	5.4×10^{-6}	0.16	16,656	69.4	2.1×10^{-8}
0.02	12,364	12.9	1.3×10^{-6}	0.08	9,546	19.9	1.3×10^{-9}
0.01	16,747	8.7	3.4×10^{-7}	0.04	12,478	13.0	9.0×10^{-11}
0.005	22,895	6.0	8.4×10^{-8}	0.02	16,791	8.7	2.5×10^{-11}
0.0025	38,111	5.0	2.1×10^{-8}				

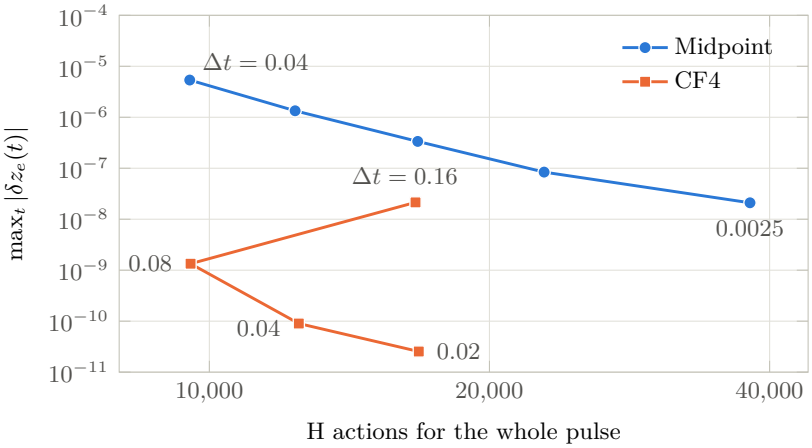


Figure 2: Dipole error against cost, from Table 6. Beyond CF4 $\Delta t = 0.08$, a longer step costs more H actions and gives a larger error.

Choosing the step. A step that needs more than about 20 H actions per exponential costs more than it saves. CF4 at $\Delta t = 0.16$ uses 69 H actions per exponential and ends up more expensive than $\Delta t = 0.04$. The cheapest point in Table 6 has $\rho\tau \approx 27$, where τ is the exponential length and ρ the spectral range. The Phase 9g runs have a Krylov limit of 20, and beyond it whole Krylov spaces are discarded. At $l_{\max} = 8$ and $\Delta t = 0.02$ ($\rho\tau \approx 45$), the sampled steps took about 49 H actions and 9 of their substeps were rejected, whereas two steps of $\Delta t = 0.01$ take about 30. A rule that sets Δt from an estimate of the spectral range would avoid this.

This explains only part of the 610 s against 116 s recorded for those length-gauge runs. H actions per unit of simulated time rise by a factor of about 1.65, but the time per H action also rises,

from 78 to 255 ms in length gauge and from 99 to 260 ms in velocity gauge, on different nodes. This did not reproduce locally: for $\Delta t = 0.02$ against $\Delta t = 0.01$, production took 506 against 480 ms/H on one core and 98 against 91 ms/H on 8 ranks. The cause is not established. The archive itself notes that it is not a controlled performance benchmark, so the Saga benchmark of Section 6 should recheck this.

2.7 Lower-order radial grids

The Krylov dimension per unit time follows the spectral range of the Hamiltonian. High-order elements have large stiffness eigenvalues, so at a fixed number of points a lower-order grid with more elements has a smaller spectral range. It also allows more ranks, because the process grid is bounded by the element counts. This is a configuration change that needs no code. Table 7 compares grids at $l_{\max} = 4$:

- The energy is insensitive to the R grid: `4x21`, `8x11` and `16x6` agree to about $10^{-7}\ E_h$. Moving R to `16x6` also leaves the dipole error unchanged, cuts the time per step by 25–30%, and raises the rank ceiling by a factor of 4.
- On the r grid at $n_r = 79$, `16x6` with R on `16x6`, against `8x11` with R on `4x21`, gives half the H actions per step, 2.4 times less time per step, a dipole error 3 times lower, and an energy error 3 times lower in magnitude. It allows 256 ranks instead of 32.
- At $n_r = 159$, `32x6` has a third of the spectral range of `16x11` (ρ_r of 3,094 against 9,764) at about twice the energy error (-6.0×10^{-7} against -2.9×10^{-7}).
- FEM-DVR quadrature is not variational, so energy errors can have either sign.

Table 7: Grid trade-offs at $l_{\max} = 4$. The energy error is against r: `32x11`, R: `32x11`. Dynamics use $E_0 = 0.05$, $\omega = 2.95$, $T = 6.4$, $\Delta t = 0.02$ on the coupled prototype, with the dipole error against r: `16x7`, R: `16x7`, whose own energy error is -1.3×10^{-5}. Grids are elements × points per element. Rank ceiling is the product of the element counts.

r	R	$n_r \times n_R$	$E - E_{\text{ref}}$	ρ (a.u.)	H/step	ms/step	$\max_t \lvert\delta z_e\rvert$	Rank ceiling
`4x11`	`4x21`	39×79	2.07×10^{-3}	687	11.6	43	5.9×10^{-5}	16
`4x11`	`16x6`	39×79	2.07×10^{-3}	614	10.3	31	5.9×10^{-5}	64
`8x11`	`4x21`	79×79	1.66×10^{-4}	2,500	21.9	185	8.0×10^{-6}	32
`8x11`	`16x6`	79×79	1.66×10^{-4}	2,441	20.1	139	8.0×10^{-6}	128
`16x6`	`8x11`	79×79	-5.06×10^{-5}	795	11.9	86	2.6×10^{-6}	128
`16x6`	`16x6`	79×79	-5.06×10^{-5}	776	11.1	76	2.6×10^{-6}	256
`32x4`	`16x6`	95×79	-1.27×10^{-4}	570	9.8	99	1.6×10^{-6}	512
`16x7`	`16x6`	95×79	-1.31×10^{-5}	1,479	14.3	148	$5.8 \times 10^{-9\dagger}$	256

† Same r grid as the reference, so this measures only the change of R grid.

2.8 Ground states

Two independent savings apply to the Davidson path:

- `track_subspace_orthogonality=True` is the default (`two_radial/diatomic.py:339`). The diagnostic computes all pairwise overlaps of the subspace at each iteration, 61,844 inner products against 8,518 without it. Turning it off left energies and iteration counts unchanged and reduced the wall time from 33.2 to 11.5 s at $l_{\max} = 6$, and from 69.4 to 32.2 s at $l_{\max} = 8$.
- The ground state lies entirely in $L = 0$, which at $l_{\max} = 8$ has 5 channels and 85 couplings. Table 8 shows that a Davidson solve restricted to $L = 0$ reproduces the archived Saga energies to 3×10^{-14} with identical H-action counts, in about half a second. The result can be transformed back to the uncoupled basis, so this path could serve production even if the dynamics stays uncoupled.

Table 8: Davidson ground state of H_2^+ restricted to $L = 0$, against the archived production energies, one core.

l_{max}	Production basis	$L = 0$ basis	$E - E_{archive}$ (Eh)	H actions (both)	Wall time, $L = 0$
2	8 / 84	2 / 6	-1.2×10^{-15}	144	0.27 s
4	29 / 1,589	3 / 19	2.3×10^{-14}	151	0.35 s
6	72 / 12,704	4 / 44	-2.8×10^{-14}	156	0.42 s
8	145 / 63,673	5 / 85	-1.8×10^{-14}	156	0.50 s

2.9 Combined effect

Table 9 combines the kernels, basis and propagator for the Phase 9g pulse at $l_{max} = 8$ on one core. The reference is the coupled basis with all L, CF4 at $\Delta t = 0.005$, and a tight Krylov tolerance ($\max_t |z_e| = 1.12 \times 10^{-3}$). Prototype times are measured propagation wall times. A complete production run at $\Delta t = 0.0025$ would take about 20 minutes, so production times are H actions multiplied by the ms/H measured over a shorter run at the same Δt.

At the same $\Delta t = 0.01$, the prototype takes the same number of H actions, has the same error, and is 20 times faster, which is the gain from the kernels, the Krylov loop, and the basis alone. CF4 then gives a better error with fewer H actions. This pulse is weak, which is why all the truncated CF4 runs have the same error, so the $L \leq 4$ and $L \leq 2$ rows do not carry over to strong fields. The all-L row has no such limit.

No attempt was made to optimize Δt for the CF4 runs. CF4 at $\Delta t = 0.10$ gave 1.8×10^{-7} with 2,486 H actions. At $\Delta t = 0.04$ the runs use about 25 H actions per exponential, which the prototype's Krylov limit of 30 allows. With the limit of 20 used in Phase 9g, this step would trigger refinement, so the limit should be raised or the step shortened.

Table 9: End-to-end comparison on one core, Phase 9g pulse, $l_{max} = 8$. Speed-up is against production at $\Delta t = 0.0025$, the production run with comparable error.

Configuration	Δt	H actions	Time (s)	$\max_t \lvert\delta z_e\rvert$	Speed-up
Production, midpoint	0.01	1,452	686	5.70×10^{-8}	
Production, midpoint	0.005	1,893	859	1.42×10^{-8}	
Production, midpoint	0.0025	2,655	1,168	3.56×10^{-9}	1
Prototype, all L, midpoint	0.01	1,452	34	5.70×10^{-8}	
Prototype, all L, CF4	0.04	1,239	29	4.62×10^{-9}	40
Prototype, $L \leq 8$, CF4	0.04	1,239	21	4.62×10^{-9}	57
Prototype, $L \leq 4$, CF4	0.04	1,239	8.7	4.62×10^{-9}	134
Prototype, $L \leq 2$, CF4	0.04	1,161	3.3	4.62×10^{-9}	354

3 `two_radial`: parallelization

3.1 Current strong scaling

Table 10 gives production's strong scaling for complete propagation at $l_{max} = 8$. Balance limit is the efficiency that the largest tile alone allows. Both coordinates have 4 elements, so a factor of 3 in the process grid leaves one rank with two elements: 2×3 and 3×4 barely beat 2×2 and 2×4. Process grids should therefore divide the element counts. A local Allreduce costs about 2 µs here, so the time in reductions is almost entirely ranks waiting for slower ones.

Table 10: Production strong scaling at $l_{\max} = 8$ on the Phase 9g grid, shared-memory MPI. Grids are $P_r \times P_R$.

Grid	Ranks	Largest tile	Balance limit	ms/H	Efficiency	Time in reductions
1×1	1	100%	1.00	473	1.00	0%
1×2	2	50.6%	0.99	272	0.87	2%
2×2	4	26.0%	0.96	158	0.75	23%
2×3	6	25.3%	0.66	148	0.53	42%
2×4	8	13.0%	0.96	90	0.66	24%
3×4	12	12.3%	0.68	83	0.48	45%

3.2 What limits scaling

Memory. The distributed Hamiltonian wraps a complete serial Hamiltonian on every rank (`two_radial/distributed_2d_hamiltonian.py:127`). That object always builds `h2_transpose`, `_h1_pairs` and `_h2_pairs_transpose` (`two_radial/hamiltonian.py:35`, `:68`), which only the global radial mode uses. Table 11 shows the result: retained memory is about 800 times the tile at $l_{\max} = 16$.

Table 11: Memory per rank (MB), measured for rank 0. The radial operator arrays are not used by the element-local action.

	$l_{\max} = 8$, 128 ranks r: `32x11`, R: `16x11`	$l_{\max} = 16$, 512 ranks r: `64x11`, R: `32x11`
`_h1_pairs`	216	5,596
`_h2_pairs_transpose`	54	1,395
Dense `h1`, `h2`, `h2_transpose`	84	1,194
Coupling arrays and table	4	279
Radial multipoles (two copies)	8	56
Retained per rank	676	9,006
Peak per rank	710	10,090
Python and libraries alone	265	264
Owned state tile, one vector	1.7	11

Setup. `build_coulomb_couplings` (`two_radial/couplings.py:49`) is a Python loop that calls sympy for Gaunt coefficients at 0.77 ms each, and every rank repeats it. This took 82 s at $l_{\max} = 12$ and up to 3 minutes at $l_{\max} = 16$, with a 2.2 GB peak. Building once on the root and broadcasting the arrays removes both. grid-lib already supports `py3nj`, which is not installed here. A quick floating-point Racah formula differed by up to 2.4×10^{-8} in relative terms, so it is not an adequate replacement.

Rank ceiling. Tiles cannot be smaller than one element, so the rank count cannot exceed the product of the element counts: 16 on the Phase 9g grid. Section 2.7 shows how to raise it without losing accuracy.

Krylov reductions. Real-time propagation issues 8–13 Allreduce calls per H action, against 2 in the lean loop (Section 2.2). The cost is small on shared memory and unmeasured on a real network.

Imports. Each rank imports 1,784 modules from a 24,685-file environment. Of these, 36% come from sympy, mpmath, matplotlib and coverage. The sympy and matplotlib imports are at the top level of grid-lib (`spherical_coordinates/angular_momentum.py:2`, `pseudospectral_grids/femdvr.py:8`), and Numba imports coverage whenever it is installed. This probably contributes

to the transient import failures on Saga: coverage is the module named in the Phase 3 failure, and Phase 9g lost two jobs to NumPy and Numba imports.

State I/O. Distributed checkpoints still gather to rank zero, as the plan notes. Phase 10 remains necessary for large runs. An $L = 0$ initial state is small, which relieves the root during setup.

3.3 Improved kernels under MPI

The MPI prototype has the lean Krylov loop and the Numba kernels but keeps the uncoupled basis. It uses the same exchange pattern as production, and its action matches production to 8.9×10^{-16} on the 1×1, 1×2, 2×1 and 2×2 grids. Table 12 compares the two under the same conditions. The prototype stays 3 to 4.5 times faster at every rank count. Its efficiency is similar to production's, so the single-core gain carries over as far as this machine can test. The faster kernels do not help unbalanced grids, so process grids should still divide the element counts.

Table 12: Production against the MPI prototype on the Phase 9g grid, shared-memory MPI, load-gated quiet machine, best of two accepted runs. Reduction time is the share of runtime in production's Allreduce calls. Exchange time is the share in the prototype's halo and weak-form exchanges.

			Production			Prototype			
$l_{\max}$	Grid	Ranks	ms/H	Efficiency	Reductions	ms/H	Efficiency	Exchange	Gain
8	1×1	1	473	1.00	0%	116	1.00	0%	4.1
8	1×2	2	272	0.87	2%	68.4	0.85	1.5%	4.0
8	2×2	4	158	0.75	23%	40.3	0.72	3.6%	3.9
8	2×4	8	90.1	0.66	24%	24.0	0.60	10.6%	3.8
8	3×4	12	83.0	0.48	45%	22.5	0.43	8.9%	3.7
4	1×1	1	26.9	1.00	0%	8.06	1.00	0.1%	3.3
4	2×2	4	7.92	0.85	17%	2.60	0.78	15.8%	3.0
4	2×4	8	4.69	0.72	19%	1.30	0.77	14.7%	3.6
4	3×4	12	4.45	0.50	36%	1.22	0.55	17.7%	3.6
2	1×1	1	4.09	1.00	0%	1.17	1.00	0.3%	3.5
2	2×2	4	1.79	0.57	12%	0.44	0.66	9.6%	4.0
2	2×4	8	1.42	0.36	15%	0.32	0.46	13.8%	4.4

These efficiencies may understate what a server node can do. A triad benchmark on this machine gives 22.8 GB/s on one thread and saturates at about 27 GB/s from four threads. This desktop CPU has two memory channels, whereas Saga's CPUs have six each. At $l_{\max} = 8$, the losses at 2×2 and 2×4 are not explained by tile balance or exchange time. Memory bandwidth is a plausible cause, but this was not established, and the Saga benchmark should show how much of the loss remains.

3.4 Clusters

Table 13 summarizes the relevant Sigma2 documentation [3, 4, 5, 6, 7, 8, 9, 10].

- **Saga** is the validation target. The current environment already works there, and a benchmark of the improved code should run there first to obtain real network and efficiency numbers.
- **Betzy** needs at least 512 tiles, for example 32 by 16 elements, and the memory fix. Its `devel` queue allows testing on 1 to 4 nodes. A grid with fewer than 512 tiles would leave allocated cores idle in the normal queue.
- **Olivia** requires a container environment (HPC-container-wrapper or Apptainer), which also removes the file-count problem. A `small` job on one node avoids the network entirely and

Table 13: Sigma2 clusters, CPU partitions. Memory is the usable amount for Saga and Betzy and the listed amount for Olivia.

	Saga	Betzy	Olivia
CPU	Xeon Gold 6138 / 6230R	EPYC 7742	EPYC Turin
Cores per node	40 or 52	128	256
Memory per node	178.5 GiB	244 GiB	768 GiB
Memory per core	3.4–4.5 GiB	1.9 GiB	3 GiB
Network	not stated in the documentation	InfiniBand HDR100, Dragonfly+	Slingshot, 200 Gb/s per node
Job size	up to 256 billing units	whole nodes, 4 to 512 nodes (at least 512 cores)	`small`: up to 256 billing units on one node; `large`: whole nodes, up to 9
Test queue	`devel`: 128 units, 2 h	`devel`: 1 to 4 nodes, 60 min	`devel`: 1152 units, 2 h
Software	Conda, as used so far	not assessed	containers required; native pip and conda not allowed

could be enough for many runs after the efficiency work. OpenMPI settings differ between single-node and multi-node jobs. Each node has 3.84 TB of local NVMe, which may suit checkpoints under Phase 10. Olivia also has GH200 GPU nodes, which were not assessed.

3.5 Projection to large calculations

A simple model projects larger cases, with costs per coupling-point and per channel-point:

$$T_{\text{production}} = 0.96\,\text{ns} \cdot \text{couplings} \cdot n_r n_R + 198\,\text{ns} \cdot \text{channels} \cdot n_r n_R, \tag{4}$$

$$T_{\text{lean}} = 0.26\,\text{ns} \cdot \text{couplings} \cdot n_r n_R + 38\,\text{ns} \cdot \text{channels} \cdot n_r n_R. \tag{5}$$

The lean coefficients are fitted to the uncoupled and all-L measurements at $l_{\max} = 8$. They overestimate the $L \le 8$, $L \le 4$ and $L \le 2$ rows of Table 1 by 3%, 10% and 22%. Table 14 applies the model at $l_{\max} = 16$ with a Saga-core factor of 2.8. The parallel column assumes ideal scaling on 512 cores with no communication, so it is a lower bound. On grid B, 10^5 H actions on 512 cores would take at least 20 days with production's kernels and at least 1.7 hours with the lean kernels, coupled basis, and $L \le 8$. In practice production would not fit in memory on grid B, because the unused `_h1_pairs` alone would need about 22 GB per rank.

Table 14: Model projection for $l_{\max} = 16$: state-vector size, time per H action on one Saga core, and ideal time per H action on 512 Saga cores. Grid A has r: `64x6`, R: `32x6` (319×159). Grid B has r: `256x6`, R: `64x6` (1279×319). Not measured.

	Grid A			Grid B		
Configuration	GB	s/H	ms/H on 512	GB	s/H	ms/H on 512
Production	1.39	1,104	2,156	11.18	8,880	17,345
Lean kernels, uncoupled	1.39	295	577	11.18	2,375	4,638
Lean, coupled, all L	0.76	13.1	26	6.09	106	206
Lean, coupled, $L \le 8$	0.24	3.9	7.6	1.93	31	61
Lean, coupled, $L \le 4$	0.09	1.3	2.5	0.74	10	20

3.6 Threads, angular decomposition and GPUs

Thread scaling of the Coulomb kernel alone is good: efficiency is 0.85 at 8 threads and 0.80 at 12 for production's kernel, and 0.77 and 0.69 for the new one, which stays about 3 times faster

throughout. The radial operators and the Krylov loop are not threaded. Phase 6, which ran Numba on one thread and measured only the Hamiltonian, found pure MPI fastest at every core count tested. The plan defers hybrid execution until radial tiling is exhausted, and this assessment found no reason to change that. Threads would become relevant if memory per rank or the element count becomes limiting, and they avoid the tile-balance constraint.

The coupled basis would also allow distribution over L as a third dimension: the Coulomb term is local to each L block, the radial operators are local to each channel, and the field couples only neighboring L. This was not tested, and the plan also defers distributing angular channels. GPUs were not assessed.

4 `one_radial`

`one_radial` describes one electron in a clamped-nuclei diatomic potential. There is no time-propagation path, only diagonalization, so the gains are in the solver and the H action.

Table 15: `one_radial` ground state on one core. The H action compares `apply_fixed_m` with the prototype. Dense diagonalization was not attempted at $N = 41{,}419$.

				Ground-state solve			H action (ms)	
$r_{\max}$	p	$l_{\max}$	N	Dense	Shift-invert	Davidson	Now	Fast
16	21	20	2,079	0.48 s	0.06 s	6 ms	0.95	0.03
16	21	60	6,039	13.8 s	1.05 s	11 ms	9.7	0.08
128	11	30	3,689	3.2 s	0.06 s	8 ms	3.1	0.04
1024	11	60	41,419		0.81 s	0.11 s	229	1.0

- **H action.** `apply_fixed_m` rebuilds the dense potential on every call, through `fixed_m_matrix` in grid-lib (`one_radial/hamiltonian.py:251`), and uses a dense second derivative (`:240`). Caching the blocks and using sparse products gives a factor of 33 at $l_{\max} = 20$, 123 at $l_{\max} = 60$, and 226 at $r_{\max} = 1024$. The potential is only 11–39% nonzero, depending on the grid.
- **Solver.** `one_radial/ground_state.py:66` calls dense `eigh`. The existing Davidson solver with a one-body spectral preconditioner converges in 8 H actions in all four cases. Where dense results exist, Davidson and shift-invert agree with each other and differ from `eigh` by at most 6.2×10^{-10} E_h, not far above the conditioning floor of about 10^{-10} E_h described in `docs/one_electron_diatomic_validation.md`. Which value is closest to the exact result was not established.
- **Setup.** At $l_{\max} = 60$ the constructor takes 22–28 s from a cold start, which is longer than the dense solve and would dominate a run once the solver is fast. It uses the same sympy Gaunt path as `two_radial`, so the same remedies apply, and they are needed to realize the full gain.
- **Symmetry.** For $Z_1 = Z_2$ the even-l channels decouple, which about halves the dimension.
- **Parallelism.** Parameter scans suit Slurm job arrays, one job per geometry. A future TDSE is small: with $N = 41{,}419$ a state vector is 0.66 MB and an H action takes 1.0 ms on one core. With a z-polarized field the m blocks are independent. It should fit on one node with threads or a one-dimensional radial decomposition, and Betzy's 4-node minimum would be a poor match.

5 Options tested and not recommended

- **C kernels.** Numba reached a factor of 3.85 against 4.0 for clang-compiled C.
- **Capping the centrifugal term.** At $l_{\max} = 8$, a cap at 2000 a.u. added an error of 7.2×10^{-8}

and increased H actions from 2,938 to 3,109. At $l_{\max} = 4$, a cap at 200 a.u. reduced H actions per step from 11.7 to 10.0 at a state error of 2.8×10^{-5}.

- **Tuning the tile size of the current kernel.** It has no measurable effect, because the tile loop sits inside the loop over output channels.
- **Faster observable sampling.** One sample costs 1.4–1.7 H actions, which is about 1% at every 10 steps and 11% at every step.
- **Faster layout construction.** It takes 0.09 s at 8,192 ranks.
- **Multipole screening** is at most a later option. At $\epsilon = 10^{-12}$ and tile 64 it would skip almost nothing at $l_{\max} = 8$ on the Phase 9g grid, and 37% at $l_{\max} = 16$ on a 319×159 grid.

6 Recommended order of work

Effort figures are rough estimates. Items 1 to 4 do not depend on the decision about the coupled basis.

Table 16: Recommended order of work.

	Work	Effort	Expected effect
1	Quick fixes: `track_subspace_orthogonality` off by default; no unused arrays in element-local mode; couplings built on the root and broadcast; `inner_products` wired into real time and the lean Krylov loop; process grids that divide element counts, with a warning otherwise; low-order grid recipe; no top-level sympy or matplotlib imports in grid-lib, no coverage in run environments	days	Davidson 2–3 times faster; memory much closer to tile size; 2 collectives per H action; 1.2 times faster per H action; up to 8 times more ranks on the same number of points
2	Numba kernels for Coulomb, radial and dipole terms	about a week	3.4 times faster per H action at $l_{\max} = 8$ (390 to 116 ms)
3	CF4 and a step rule based on the spectral range, with velocity-gauge and absorber validation	days	4,000 to 15,000 times lower error at equal H actions
4	`one_radial`: sparse H action and Davidson as default	1–2 days	Up to 226 times faster H action; solve more than 1000 times faster, once constructor cost is also fixed
5	Coupled basis: tables, embedding, archives, non-scalar observables	several weeks	5 times faster per H action at $l_{\max} = 8$ with all L, growing with $l_{\max}$; more with truncation
6	Scale out: Saga benchmark, container environment and Phase 10 I/O, then Betzy and Olivia	follows 1–2	Real network and efficiency numbers

7 Limitations

- **One machine.** All timings come from a local workstation with shared-memory MPI, up to 12 ranks, and strongly limited memory bandwidth. Saga times are scaled by a measured core factor. Network cost, parallel file systems, and server memory hierarchies were not measured.
- **Model projections.** Section 3's projection assumes ideal scaling and extrapolates from small grids and $l_{\max} \leq 8$.
- **Truncation in L** was tested only at $l_{\max} = 6$ in a 16 a.u. box without an absorber. Convergence in L for strong-field ionization needs its own physics validation.
- **CF4** was run in length gauge only. Velocity gauge is also linear in the field, so the same scheme should apply, but neither that nor an absorber has been tested.

- **Low-order grids** were tested at $l_{\max} = 4$ for a short pulse. They should be rechecked for high-energy photoelectrons and for dissociation.
- **Removing reorthogonalization** was tested on short runs with Krylov dimensions up to about 15.
- **Combined timings** for production are H actions multiplied by ms/H from shorter runs, not wall times of complete runs.
- **Prototypes** do not yet cover absorbers, archives, or non-scalar observables in the coupled basis, and the coupled basis has run only on one rank: the MPI prototype is uncoupled. GPU execution was not assessed.

A Side findings

These are not performance issues, but they came up during the assessment.

- `two_radial/state_archive.py:539` requires exact equality of the stored radial points and weights. A state written on Saga with NumPy 2.5.2 was rejected locally with NumPy 2.5.4, which will also block moving states between clusters. A tight tolerance would keep the safety check.
- `sample_initial=False` is ignored when a sampling interval is set, because 0 mod interval = 0 (`two_radial/dynamics_sampling.py:325`).
- The plan states that Krylov orthogonalization batches basis projections (`dev/improved_parallelization_plan.md:142`). This holds for the ground-state path but not for real time, where `inner_products` is not passed.
- The plan states that tile size 512 was selected (line 648), whereas the code default is 2048 (`two_radial/hamiltonian.py:31`). No difference was measured between them in the current kernel.
- grid-lib issues an overflow warning at $r_{\max} = 1024$ and high L (`spherical_coordinates/potentials.py:175`).

B Software

The installed `grid-tdse` environment was sufficient for the assessment, and nothing was added to it. Two optional packages would be useful later: `py3nj` for faster Gaunt coefficients, and an MPI-enabled `h5py` if Phase 10 chooses HDF5.

References

[1] S. Blanes and P. C. Moan, Fourth- and sixth-order commutator-free Magnus integrators for linear and non-linear dynamical systems, *Appl. Numer. Math.* **56**(12), 1519–1537 (2006). doi:10.1016/j.apnum.2005.11.004

[2] A. Alvermann and H. Fehske, High-order commutator-free exponential time-propagation of driven quantum systems, *J. Comput. Phys.* **230**(15), 5930–5956 (2011). doi:10.1016/j.jcp.2011.04.006

[3] Sigma2 documentation, Saga. https://documentation.sigma2.no/hpc_machines/saga.html

[4] Sigma2 documentation, Job types on Saga. https://documentation.sigma2.no/jobs/job_types/saga_job_types.html

[5] Sigma2 documentation, Betzy. https://documentation.sigma2.no/hpc_machines/betzy.html

[6] Sigma2 documentation, Job types on Betzy. https://documentation.sigma2.no/jobs/job_types/betzy_job_types.html

[7] Sigma2 documentation, Olivia. https://documentation.sigma2.no/hpc_machines/olivia.html

[8] Sigma2 documentation, Job types on Olivia. https://documentation.sigma2.no/jobs/job_types/olivia_job_types.html

[9] Sigma2 documentation, Installing and using software on Olivia. https://documentation.sigma2.no/hpc_machines/olivia/software_stack.html

[10] Sigma2 documentation, OpenMPI on Olivia. https://documentation.sigma2.no/hpc_machines/olivia/openmpi.html